

Dry Run

① ArrayBasis.java



1st Array creation

```
int marks[] int [] marks = new int [5];  
marks = [0, 0, 0, 0, 0];  
String [] names = new String [3];  
names = [null, null, null]  
marks.length = 5
```

output:-

int defaults : [0, 0, 0, 0, 0]
String defaults : [null, null, null]
length is fixed = 5

2nd Array a

```
a = [10, 20, 30, 40, 0, 0, 0, 0]  
Index = 0, 1, 2, 3 4 5 6 7  
size = 4  
a[2] = 30  
o/p a[3] = 30
```

3rd Insert 15 at index 1

shifts = 0

size = 4

index = 1

value = 15

insertAt (a, 4, 1, 15)

$i = \text{size} = 4$

$4 > 1$ true

$a[4] = a[3]$

$a[4] = 40$

shifts++

shifts = 1

now,

$i--$

$i = 3$

$a = [10, 20, 30, 40, 40, 0, 0, 0]$

$i = 3$

$3 > 1$ true

$a[3] = a[2]$

$a[3] = 30$

shifts++ so shifts = 2

$i--$ so $i = 2$

$a = [10, 20, 30, 30, 40, 0, 0, 0]$

$i = 2$

$2 > 1$ true

$a[2] = a[1]$

$a[2] = 20$

shifts++ so shifts = 3

$i--$ so $i = 1$

$a = [10, 20, 20, 30, 40, 0, 0, 0]$

$i = 1$

$i > 1$ False, Loop stops
now, $a[\text{index}] = \text{value}$

$a[1] = 15$

$a = [10, 15, 20, 30, 40, 0, 0, 0]$

at main, size++

size = $4 + 1 = 5$

shifts = 3

output: Insert 15 at 1: $[10, 15, 20, 30, 40, 0, 0, 0]$

shifts = 3

4th Insert 50 at end

shifts = 0

size = 5

index = 5

value = 50

insertAt($a, 5, 5, 50$)

$i = 5$

$5 > 5$ false loop does not execute

$a[5] = 50$

$a = [10, 15, 20, 30, 40, 50, 0, 0]$

size = 6

5th delete index 0

shifts = 0

$i = 0$, $0 < 5$, $a[0] = a[1] = 15$, shifts = 1, $i++$

$i=1$ $1 < 5 \checkmark$ $a[1] = a[2] = 20$ shifts = 2 $i++$
 $i=2$ $2 < 5$ $a[2] = a[3] = 30$ shifts = 3 $i++$
 $i=3$ $3 < 5$ $a[3] = a[4] = 40$ shifts = 4 $i++$
 $i=4$ $4 < 5$ $a[4] = a[5] = 50$ shifts = 5 $i++$
 $i=5$ $5 < 5 \times$

$a[5] = 20$

$a = [15, 20, 30, 40, 50, 0, 0, 0]$

size = 5

Bound check: $a[8] = 99$ ArrayIndexOutOfBoundsException

2-D Array

~~r=0~~ $r=0 \rightarrow [1]$

$r=1 \rightarrow [1, 1]$

$r=2 \rightarrow [1, 2, 1]$

$r=3 \rightarrow [1, 3, 3, 1]$

Copy of $a.length = 8$
 $8 \times 2 = 16$

bigger = $[15, 20, 30, 40, 50, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$

② ArrayStack.java

Initial:

capacity = 4

top = -1

stack = $[]$

push(10): $++top \rightarrow 0$
data $[0] = 10$

stack = [10]

push(20):

++top $\rightarrow$ 1

data[1] = 20

stack = [10, 20]

push(30):

++top $\rightarrow$ 2

data[2] = 30

stack = [10, 20, 30]

peek(): data[top] = data[2] = 30
peek = 30

pop():

item = data[2] = 30

top -- $\rightarrow$ 1

stack = [10, 20]

push(40):

++top $\rightarrow$ 2

data[2] = 40

stack = [10, 20, 40]

push(50):

++top $\rightarrow$ 3

data[3] = 50

stack = [10, 20, 40, 50]

isFull:

top == 3

$3 == 3 \rightarrow \text{true}$

push(60):

full $\rightarrow$ Stack overflow

while (!isEmpty()):

pop $\rightarrow$ 50

top = 2

pop $\rightarrow$ 40

top = 1

pop $\rightarrow$ 20

top = 0

pop $\rightarrow$ 10

top = -1

isEmpty:

$-1 == -1 \rightarrow \text{true}$ loop stops

pop() again: empty $\rightarrow$ stack underflow

③ BalancedBrackets.java

Test 1

s = "a[i] = (b+c) * d;d?"

stack = []

a $\rightarrow$ ignore

[$\rightarrow$ push $\rightarrow$ [[]

i $\rightarrow$ ignore

] $\rightarrow$ pop [$\rightarrow$ matches [] $\rightarrow$ true

= $\rightarrow$ ignore

($\rightarrow$ push $\rightarrow$ [(]

b → ignore
 + → ignore
 c → ignore
) → pop (→ matches () → true
 * → ignore
 d → push → [d]
 d → ignore
 { → pop d → matches { } → true

Stack.isEmpty() → true
 Result = balanced

Test 2

s = "{ [()] }"
 Stack = []

{ → push → [{]
 [→ push → [{, []
 (→ push → [{, [, (
) → pop (→ match true
 (→ push → [{, [, (
) → pop → match true
] → pop → match true
 } → pop → match true

Stack.isEmpty() → true
 Result = balanced

Test 3

$s = "[([)]] "$

stack = []

($\rightarrow$ push $\rightarrow$ [(]

[$\rightarrow$ push $\rightarrow$ [(, []

) $\rightarrow$ pop [

matches [and]

$\rightarrow$ false

Result = not balanced

Test 4

$s = "((a + b)) "$

stack = []

($\rightarrow$ push $\rightarrow$ [(]

($\rightarrow$ push $\rightarrow$ [(, (]

a $\rightarrow$ ignore

+ $\rightarrow$ ignore

b $\rightarrow$ ignore

) $\rightarrow$ pop ($\rightarrow$ match true

stack = [(]

stack.isEmpty() $\rightarrow$ false

Result = not balanced

Test 5

$s = "a + b"$

stack = []

a = ignore

+ = ignore

$b \rightarrow \text{ignore}$
 $) \rightarrow \text{stack.isEmpty}() \rightarrow \text{true}$
 return false
 result = not balanced

Test 6

$g = 44$

stack = []

no characters to process.

stack.isEmpty() $\rightarrow$ true

Result = balanced

4. Big O Recap

Q1: $n = 1000 \rightarrow c = 500$
 $n = 2000 \rightarrow c = 1000$
 $n = 4000 \rightarrow c = 2000$
 ratio = 2

Q2:

$n = 1000 \rightarrow c = 1,000,000$
 $n = 2000 \rightarrow c = 2,000,000$
 $n = 4000 \rightarrow c = 4,000,000$
 ratio = 2

Q3:

$1000 \rightarrow 100 \rightarrow 10 \rightarrow 1 \rightarrow 0 \rightarrow c = 4$
 $2000 \rightarrow 200 \rightarrow 20 \rightarrow 2 \rightarrow 0 \rightarrow c = 4$
 $4000 \rightarrow 400 \rightarrow 40 \rightarrow 4 \rightarrow 0 \rightarrow c = 4$
 ratio = 1

Q4:

for $i=0$ to $n-1$
 j starts from i

$n=1000$:

$$C = 1000 + 999 + 998 + \dots + 1$$

$$= 500500$$

$n=2000$:

$$C = 2000 + 1999 + \dots + 1$$

$$= 2001000$$

$n=4000$:

$$C = 4000 + 3999 + \dots + 1 = 8002000$$

$$\text{ratio} = 2001000 / 500500 \approx 4$$

Q5:

outer loop : $i=n, n/2, n/4, \dots$

inner loop : n times

$n=1000$:

outer iterations $= 10$

$$C = 10 \times 1000 = 10,000$$

$n=2000$:

outer iterations $= 11$

$$C = 22000$$

$n=4000$:

outer iterations $= 12$

$$C = 48000$$

$$\text{ratio} = 22000 / 10000 = 2.2$$

Q6: $i = 1, 2, 4, 8, \dots$
for each i , inner loop runs i times

$$n = 1000$$

$$1 + 2 + 4 + \dots + 512 = 1023$$

$$n = 2000$$

$$1 + 2 + 4 + \dots + 1024 = 2047$$

$$n = 4000$$

$$1 + 2 + 4 + \dots + 2048 = 4095$$

$$\text{ratio} = 2047 / 1023 \approx 2$$

Q7: outer loop $= n$
inner loop $= \sqrt{n}$

$$n = 1000:$$

$$c = 1000 \times 31 = 31000$$

$$n = 2000:$$

$$c = 2000 \times 44 = 88000$$

$$n = 4000:$$

$$c = 4000 \times 63 = 252000$$

$$\text{ratio} = 88000 / 31000 \approx 2.84$$

Q8:

j takes multiples of i

$$n = 1000:$$

$c =$ number of multiples for each i

$= 1000/i$ summed for $i = 1$ to 1000

$n = 2000:$ same process

$n = 4000:$ same process

ratio approx 2

Recursive calls

twoCalls(n): calls = $2^n - 1$

~~n=10~~ n=10 → 1023

n=11 → 2047

n=12 → 4095

n=20 → 1,048,575

oneCall(n):

calls = n

n=10 → 10

n=11 → 11

n=12 → 12

n=20 → 20

5. Circular Deque

Initial: front = 0, size = 0

data = [_, _, _, _]

addLast(B):

index = 0 → B

size = 1

[B, -, -, -]

addLast(C):

index = 1 → C

size = 2

[B, C, -, -]

addFirst (A):

$$\text{front} = (0 - 1 + 4) \% 4 = 3$$

data [3] = A

size = 3

[B, C, -, A]

addFirst (Z):

$$\text{front} = (3 - 1 + 4) \% 4 = 2$$

data [2] = Z

size = 4

[B, C, Z, A]

addLast (D):

full $\rightarrow$ grow

data = [Z, A, B, C, -, -, -, -]

front = 0

D $\rightarrow$ index 4

size = 5

[Z, A, B, C, D, -, -, -]

removeFirst:

Z removed

front = 1, size = 4

removeLast:

D removed size = 3

Remaining = [A, B, C]

Stack

addFirst : 1 $\rightarrow$ 2, 1 $\rightarrow$ 3, 2, 1

removeFirst : 3 $\rightarrow$ 2 $\rightarrow$ 1

Queue

addlast: $1 \rightarrow 1, 2 \rightarrow 1, 2, 3$

removefirst: $1 \rightarrow 2 \rightarrow 3$

off Stack: 3 2 1

Queue: 1 2 3

6. Circular Queue

Initial: Front = 0, size = 0

data = [-, -, -, -, -]

enqueue 10: $\text{rear} = (0 + 0) \% 5 = 0$

data[0] = 10

size = 1

enqueue 20: $\text{rear} = (0 + 1) \% 5 = 1$

data[1] = 20

size = 2

enqueue 30: $\text{rear} = 2$, data[2] = 30, size = 3

enqueue 40: $\text{rear} = 3$, data[3] = 40, size = 4

enqueue 50: $\text{rear} = 4$, data[4] = 50, size = 5

data = [10, 20, 30, 40, 50]

front = 0, size = 5

full = true

3 Dequeue: 10 removed

front = $(0 + 1) \% 5 = 1$

size = 4

dequeue: 20 removed
 $\text{front} = (1+1) \cdot 1.5 = 2$
 $\text{size} = 3$

dequeue: 30 removed
 $\text{front} = (2+1) \cdot 1.5 = 3$
 $\text{size} = 2$

$\text{data} = [-, -, -, 40, 50]$
 $\text{front} = 3$

enqueue 60
 $\text{rear} = (3+2) \cdot 1.5 = 0$
 $\text{data}[0] = 60$
 $\text{size} = 3$

$\text{data} = [60, -, -, 40, 50]$

enqueue 70
 $\text{rear} = (3+3) \cdot 1.5 = 1$
 $\text{data}[1] = 70$
 $\text{size} = 4$

$\text{data} = [60, 70, -, 40, 50]$

enqueue 80
 $\text{rear} = (3+4) \cdot 1.5 = 2$
 $\text{data}[2] = 80$
 $\text{size} = 5$

$\text{data} = [60, 70, 80, 40, 50]$
 $\text{full} = \text{true}$

FIFO order: $\text{front} \rightarrow 40, 50, 60, 70, 80 \leftarrow \text{rear}$

enqueue 90 if full: $\text{size} \geq \text{data.length}$
 $5 \geq 5 \rightarrow \text{true}$

Queue overflow

7. Dynamic Array

Initial: capacity = 4 size = 0 data = [-, -, -, -]

add () - 1 to 20

10 → size = 1

20 → size = 2

30 → size = 3

40 → size = 4

data = [10, 20, 30, 40] capacity = 4

Grow Add 50: size == capacity, 4 == 4 → true

grow: oldCap = 4

newCap = $4 + 4/2 = 6$

copies = 4

size = 8

add 60

size = 6

add 70

grow: 6 → 9, copies = 6, size = 9

add 80, 90 size = 9

add 100 grow 9 → 13, copies = 9, size = 10

add 110, 120, 130
size = 13

add 140
grow : 13 $\rightarrow$ 19
copies = 32
size = 14

add 150, 160, 170, 180, 190 size = 19

add 200, grow : 19 $\rightarrow$ 28, copies = 51, size = 20

final : size = 20, capacity = 28, copies = 51

removeAt(0): 10 removed remaining starts from 20
size = 19

amortised 1000 $\rightarrow$ 2, 3 copies

10,000 $\rightarrow$ 2, 3 copies

add() = amortised $O(1)$

8. FrontInsertRate

$n = 25,000$

~~Arraylist :~~

~~add(0, x) $\rightarrow O(n^2)$~~

ArrayList.add(0, x) $\rightarrow$ shifting $\rightarrow O(n^2)$

LinkedList.addFirst(x) $\rightarrow O(1)$

ArrayDeque.addFirst(x) $\rightarrow O(1)$

$N = 50,000$

ArrayList $\rightarrow O(N^2)$

LinkedList $\rightarrow O(1)$

ArrayDeque $\rightarrow O(1)$

$N = 100,000$

ArrayList $\rightarrow O(N^2)$

LinkedList $\rightarrow O(1)$

ArrayDeque $\rightarrow O(1)$

$N = 200,000$

ArrayList $\rightarrow O(N^2)$

LinkedList $\rightarrow O(1)$

ArrayDeque $\rightarrow O(1)$

overall

ArrayList $\rightarrow O(N^2)$

LinkedList $\rightarrow O(N)$

ArrayDeque $\rightarrow O(N)$

9. Java Queues Demo

① ArrayDeque as stack

push page1 $\rightarrow$ [page1]

push page2 $\rightarrow$ [page1, page2]

push page3 $\rightarrow$ [page3, page2, page1]

pop → page 3
 pop → page 2
 output : page 3 then page 2

2. ArrayDeque as Queue

offer job1 → [job1]
 offer job2 → [job1, job2]
 offer job3 → [job1, job2, job3]

poll → job1
 poll → job2
 output : job1 then job2

3. Deque

offerLast(2) → [2]
 offerFirst(1) → [1, 2]
 offerLast(3) → [1, 2, 3]

pollLast → 3
 pollLast → 2
 Remaining : [1]

4. Empty Queue

poll() → null
 peek() → null
 remove() → NoSuchElementException

5. offer(null)

→ NullPointerException

6. ArrayBlockingQueue (capacity=2)

offer(1) → true

offer(2) → true

offer(3) → false

Queue = [1, 2]

10. Linear Queue

front = 0 rear = -1

data = [-, -, -, -, -]

enqueue 10 → rear = 0

enqueue 20 → rear = 1

enqueue 30 → rear = 2

enqueue 40 → rear = 3

enqueue 50 → rear = 4

data = [10, 20, 30, 40, 50]

front = 0 rear = 4

dequeue 10 → front = 1

dequeue 20 → front = 2

dequeue 30 → front = 3

data = [0, 0, 0, 40, 50]
 front = 3 rear = 4
 enqueue 60

rear == data.length - 1

4 == 4 → true

queue overflow

3 slots are empty but rear is already at last index and linear queue cannot reuse empty slots so circular queue fixes this problem

11. Postfix Evaluation

1) $3 + 4 * 2$

postfix $3 4 2 * +$

3

$4, 2 \rightarrow 4 * 2 = 8$

$3, 8 \rightarrow 3 + 8 = 11$

2) ~~$(3 + 4) * 2$~~

~~postfix $3 4 + 2 *$~~

~~evaluation $3 + 4 = 7$~~

~~$7 * 2 = 14$~~

355 10-4-3

10 4 - 3 -

10-4=6

6-3=3

3 → operator? no so output 3

+ → operator? yes → precedence = 1 → push

4 → operator? no → o/p 4

* → operator? yes → precedence = 2 → push

2 → operator? no → o/p 2

pop * then +

postfix = 3 4 2 * +

result = 11

(3+4)*2

(- push

3 - o/p

+ - precedence = 1 → push

4 - output

) - pop +

* - precedence = 2 → push

2 - o/p

so 3 4 + 2 *

output 14

10-4-3

- $\rightarrow$ precedence = 1

Second - $\rightarrow$ same precedence $\rightarrow$ pop first -

postfix = 10 4 - 3 -
result = 3

2^3^2

$\wedge \rightarrow$ precedence = 3

Second $\wedge \rightarrow$ same precedence, don't pop

postfix = 2 3 3 $\wedge \wedge$
result = 512

$(5+3) * (12/4) - 7$

postfix = 5 3 + 12 4 / * 7 -
result = 17

12. Singly linked list

addlast(20) $\rightarrow$ 20

addlast(30) $\rightarrow$ 20 $\rightarrow$ 30

addfirst(10) $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 30

add(2, 25) $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 25 $\rightarrow$ 30

addlast(40) $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 25 $\rightarrow$ 30 $\rightarrow$ 40

index of (30) = 3, get(3) = 30, size = 5

remove(25) → 10 → 20 → 30 → 40

removeFirst() → 10

removeLast() → 40 → 20 → 30

addLast(50), addLast(60)

→ 20 → 30 → 50 → 60

reverse() → 60 → 50 → 30 → 20

sum = 160

Queue

① Circular Queue Demo

front = -1 rear = -1

queue = [-, -, -, -, -]

enqueue(10) → front = 0 rear = 0
[10, -, -, -, -]

enqueue(20) rear = 1

[10, 20, -, -, -]

enqueue(30)

[10, 20, 30, -, -]

dequeue() = 10 front = 1

enqueue(40) → rear = 3

enqueue(50) → rear = 4

[10, 20, 30, 40, 50]

front = 1 rear = 4

dequeue() → 20 front = 2

dequeue() → 30 front = 3

enqueue(60)

rear = 0 [60, -, -, 40, 50]

front = 3 rear = 0

o/p 40 50 60

isfull() (rear + 1) % max == front

⇒ (0 + 1) % 5 = 1 ≠ 3

⇒ false

② Q Demogeneric

front = -1 rear = -1

A → B dequeue A remaining B

enqueue 10 → [10]

enqueue 20 → [10, 20]

enqueue 30 → [10, 20, 30]

dequeue → 10

display → 20 30

③ Queue Demo

front = -1, rear = -1

enqueue 10 $\rightarrow$ [10]
 enqueue 20 $\rightarrow$ [10, 20]
 enqueue 30 $\rightarrow$ [10, 20, 30]
 dequeue 10
 front = 1 so off 20 30

④ Queue using linked list generic

front = null rear = null size = 0
 enqueue A
 enqueue B = A B
 enqueue C = A B C
 dequeue A, front = B
 peek = B
 display B C
 size = 2

⑤ Queue using linked list Int

front = null rear = null size = 0
 enqueue (10): front = 10 rear = 10 size = 1
 enqueue (20): rear = 20 size = 2
 enqueue (30): rear = 30 size = 3
 dequeue () removed = 10 front = 20 size = 2
 peek () = 20 display () = 20, 30

⑥ Queue using Stack demo

stackNewest = []

stackOldest = []

enqueue(10): newest = [10]

enqueue(20): newest = [10, 20]

enqueue(30): newest = [10, 20, 30]

dequeue(): [30, 20, 10]

pop() 30 10 popped

peek() = 20

⑦ Queue using Stack

inputStack = [] outputStack = []

enqueue(A):

inputStack = [A]

enqueue(B):

inputStack = [A, B]

enqueue(C):

inputStack = [A, B, C]

dequeue()

outputStack = [C, B, A]

pop(), A is popped

peek() = B

queue = B C